

ECON485 Introduction to Database Systems

Database Operations

What DBAs actually lose sleep over

Databases don't fail gracefully

- **Systems fail**
- **People make mistakes**
- **Databases obey commands, not intent**

Note that

- **SQL does exactly what you ask**
- **Most failures are self-inflicted**
- **Backup & recovery exists because humans are imperfect**

And on top of that infrastructure fails

What Can Go Wrong (In Real Life)

Typical problems

- Human error (“bad users”)
- Software bugs (“bad developers”)
- Hardware / infrastructure failure (“bad boxes”)
- Logical corruption (“bad boxes again”)
- Security issues (“bad people you may or may not know”)

What Can Go Wrong (In Real Life)

The most dangerous failures leave the database in “a working state”

The worst kinds of data problems are not the obvious crashes or errors (like constraint violations, failed transactions, or server downtime), but the silent ones where:

- The database remains fully operational**
- All constraints are satisfied**
- Queries run successfully and return results quickly**

Yet the data is wrong, inconsistent, or misleading in ways that aren't immediately detectable

What Can Go Wrong (In Real Life)

Real-World Examples

- **Silent data corruption:** A bug in application code updates salaries correctly in format and range, but applies a wrong multiplier. All CHECK constraints pass. Database is consistent and queryable — but payroll is catastrophically wrong.
- **Incorrect foreign key usage:** An order is linked to the wrong customer ID (but the ID exists). Referential integrity is preserved. The database is in a "working state," but billing goes to the wrong person.

What Can Go Wrong (In Real Life)

Real-World Examples

- **Lost updates due to poor concurrency control:** Two transactions read the same inventory count, both subtract stock, and both commit successfully (no deadlock, no error). Final stock level is wrong, but the DB never complained.
- **Gradual drift from reality:** Addresses, prices, or statuses become outdated, but never violate NOT NULL or type rules. The system runs perfectly — just on stale, inaccurate data.

Failure You Can't Detect with SQL

Imagine the following at the same time

- Data is valid**
- Data is wrong**
- Queries return results**

Failure You Can't Detect with SQL

How can data be valid and wrong at the same time?

In databases, data can be "valid" (satisfying all defined schema rules and integrity constraints) while still being "wrong" (factually inaccurate or not reflecting reality). This happens because database constraints enforce structural and syntactic correctness, but they cannot fully verify semantic accuracy—whether the data truly represents the real-world entity or event.

Validity — Data conforms to the database's rules (e.g., data types, formats, ranges, uniqueness, referential integrity). Constraints like NOT NULL, CHECK, UNIQUE, PRIMARY KEY, and FOREIGN KEY ensure this.

Accuracy — Data is correct in the real world. Constraints can't check this exhaustively, as it often requires external knowledge or verification.

Failure You Can't Detect with SQL

Database management systems are excellent at preserving consistency (as defined by ACID and constraints), but they cannot guarantee correctness in the real-world sense.

The most dangerous failures aren't the ones that break the system — they're the ones that let it keep running smoothly while lying to you.

Failure You Can't Detect with SQL

Database management systems are excellent at preserving consistency (as defined by ACID and constraints), but they cannot guarantee correctness in the real-world sense.

The most dangerous failures aren't the ones that break the system — they're the ones that let it keep running smoothly while lying to you.

Emergency Response – Backup & Recovery

Backup and recovery are critical tools for returning a database to the last known valid and correct state, but with important caveats.

They help mitigate many of the "dangerous failures" we discussed.

- A full backup (or point-in-time recovery via logs) lets you roll back to a moment when the data was verified as both valid and correct (e.g., after a successful audit, end-of-month reconciliation, or manual confirmation).**
- If bad data has been introduced gradually (wrong updates, bugs, human errors) and gone unnoticed, restoring from a clean backup wipes out the accumulated inaccuracies.**
- Since constraints only enforce validity, not real-world correctness, backups are often the only way to recover from large-scale logical/data-entry disasters.**

Emergency Response – Backup & Recovery

Backup & recovery still has limitations

- Restoring to yesterday's backup means replaying (or losing) all valid transactions since then. Point-in-time recovery (using transaction logs) reduces this window, but still requires careful application of logs up to a safe point.
- If the corruption started weeks ago and wasn't detected, your most recent "correct" backup could be far in the past — causing significant data loss or rework.
- Restoring fixes the current bad state, but if the root cause (buggy application, poor validation, untrained users) isn't fixed, the issue can reappear immediately after recovery.

Emergency Response – Backup & Recovery

Backup & recovery still has limitations

- The longer the bad data lives undetected in a "working" database, the farther back you must restore, and the higher the cost.**
- If correct data was never backed up (e.g., new records entered incorrectly from the start), recovery won't magically make them right.**

Emergency Response – Backup & Recovery

Best practices:

- **Frequent backups + transaction log backups:** Enable precise point-in-time recovery with minimal loss.
- **Regular validation and auditing:** Establish "known correct" points more frequently (e.g., daily reconciliations, checksums, business rule checks).
- **Testing restores:** Regularly verify that backups are usable.

What Is a Backup, Really?

A snapshot of the past, taken at a specific moment, blind to business context

Backup is not safety, correctness or an undo button.

A backup faithfully preserves your mistakes.

Typical Backup Types You Need to Understand

Three major types

- Full**
- Incremental**
- Logical (dump)**

Typical Backup Types You Need to Understand

A Full Backup is complete copy of the entire database at a specific point in time.

What is captured

- All data pages (tables, indexes, etc.)**
- All database structures**
- Entire content of the database, regardless of prior backups**

Typical Backup Types You Need to Understand

A Full Backup is complete copy of the entire database at a specific point in time.

What is not captured

- Changes that occur after the backup completes**
- Transaction logs (in some implementations; depends on mode)**
- Temporary objects or session-specific data**

Typical Backup Types You Need to Understand

A Full Backup is complete copy of the entire database at a specific point in time.

Trade-offs

- Time: Longest backup time; scales with database size**
- Storage: Highest storage requirements**
- Safety: Highest recovery safety — can restore independently without needing other backups**
- Complexity: Lowest complexity — simple and straightforward to manage and restore**

Typical Backup Types You Need to Understand

An Incremental Backup captures only the changes made since the last backup (full or incremental).

What is captured

- Only data pages or extents that have changed since the previous backup**
- Minimal set of modifications**

What is not captured

- Unchanged data (relies on prior backups)**
- Full database state (cannot stand alone)**

Typical Backup Types You Need to Understand

An Incremental Backup captures only the changes made since the last backup (full or incremental).

Trade-offs

- Time: Much faster backup time**
- Storage: Significantly lower storage needs**
- Safety: Good safety if chain is intact, but risk increases — losing one incremental breaks the entire chain**
- Complexity: Higher complexity — restore requires applying full + all subsequent incrementals in order**

Typical Backup Types You Need to Understand

A Logical (Dump) Backup exports the database content as a logical set of statements (e.g., CREATE, INSERT) rather than physical data files.

What is captured

- Schema definitions (tables, views, procedures, etc.)**
- Data as logical insert statements**
- Often includes permissions and metadata**

Typical Backup Types You Need to Understand

A Logical (Dump) Backup exports the database content as a logical set of statements (e.g., CREATE, INSERT) rather than physical data files.

What is not captured

- Physical storage details (file layout, page allocation)**
- Binary objects may be handled differently**
- In-flight transactions or uncommitted data**
- Performance optimizations tied to physical structure**

Typical Backup Types You Need to Understand

A Logical (Dump) Backup exports the database content as a logical set of statements (e.g., CREATE, INSERT) rather than physical data files.

Trade-offs

- Time: Can be slower for large datasets (due to generating statements)**
- Storage: Often larger than physical backups (text representation)**
- Safety: Excellent portability and resilience — can restore to different versions or platforms; less prone to physical corruption**
- Complexity: Moderate to high — restore rebuilds from scratch; good for migrations or schema changes**

Typical Backup Types You Need to Understand

Take aways

- Full backups provide the strongest foundation for recovery but are resource-intensive.
- Incremental backups optimize speed and storage but introduce dependency chains for recovery.
- Logical backups prioritize flexibility and human-readability over raw performance and size efficiency.

Full + Incremental Schemes

A structured rotation scheme combining full and incremental backups to balance recovery safety, storage efficiency, and backup time.

Uses periodic full backups as recovery anchors

Fills gaps with incremental backups (changes only)

Organizes backups into a hierarchy with retention rules (e.g., daily, weekly, monthly)

Full + Incremental Schemes

Goal:

- Fast daily backups (incrementals)
- Reasonable storage use
- Ability to recover to any recent point
- Long-term retention without exploding storage

Full + Incremental Schemes

**Grandfather-Father-Son (GFS)
Scheme: A tiered retention policy
named after family generations.**

How it works:

- **Daily: Take incremental (captures changes since last backup)**
- **Weekly: Take full (resets incremental chain)**
- **Monthly: Take another full (for long-term archive)**

Level	Type	Frequency	Retention	Role
Son	Incremental	Daily	1 week	Daily changes
Father	Full	Weekly	1 month	Complete state
Grandfather	Full	Monthly	1 year	Complete state

Full + Incremental Schemes

What is captured

- **Full backups:** Entire database at that point
- **Incrementals:** Only changes since last backup (full or incremental)
- **Together:** Ability to restore any day within retention period

Full + Incremental Schemes

What is not captured

- Changes after the last backup in the chain
- If an incremental is missing/corrupted, may lose recoverability for points after it

Full + Incremental Schemes

Trade-offs

- **Time:** Daily backups fast (incrementals); weekly/monthly slower (fulls)
- **Storage:** Much lower than full-only (reuses full as base); higher than pure incremental
- **Safety:** Excellent — multiple recovery points; can survive loss of recent incrementals by falling back to older full
- **Complexity:** Moderate — requires scheduling rotation and tracking chains; must manage expiration/deletion carefully

Full + Incremental Schemes

Key Advantages over pure strategies:

- **Faster daily operations than full-only**
- **Safer and simpler recovery than long incremental chains**
- **Built-in long-term retention**

Key Point: GFS-style schemes are a practical sweet spot — they optimize daily performance while preserving strong recovery options through periodic full resets.

Restore vs Recovery

Do not confuse these two terms

- Restore is technical action**
- Recovery is a business decision**

Walk-through scenario:

- Restore DB to yesterday**
- Business rules changed today**
- Users already acted on current state**

Restore vs Recovery

Do not confuse these two terms

- Restore is technical action: Bringing the database back to a previous point-in-time state using backups (full, incremental, logs, etc.).**
- Recovery is a business decision: Deciding what point in time to restore to, what additional steps are needed after the restore, and how to bring the system back to a correct and usable state for the business.**

Restore only gives you the data as it existed at a specific moment.

Recovery requires judgment: weighing data loss, business impact, manual corrections, and operational continuity.

Restore vs Recovery

Walk-through scenario:

- **Today (December 19, 2025): A critical bug in the application caused wrong data to be written throughout the day.**
- **The database is still "working" (no crash, constraints satisfied), but much of today's data is factually incorrect.**
- **Last clean full backup: End of day yesterday (December 18, 2025).**
- **Transaction logs available for point-in-time restore.**

Restore vs Recovery

Option 1: Simple Technical Restore

- **Action:** Restore the database to end-of-day December 18, 2025 (using yesterday's full backup + any needed logs up to that point).
- **Result:** Database now contains only correct data up to yesterday.
- All of today's changes (good and bad) are gone.

But now business problems appear

- **Business rules changed today:** Pricing rules, tax rates, or discount logic were updated this morning via a legitimate deployment. After restore to yesterday, the database schema or reference data no longer reflects the new rules.

Restore vs Recovery

But now business problems appear (cnt'd)

- Users already acted on current (wrong) state: Orders were placed, invoices generated, shipments initiated, payments processed — all based on today's buggy data. External systems (warehouses, payment gateways, customers) have been updated with wrong information. Some legitimate transactions also occurred today that need to be preserved.**

Restore vs Recovery

Actual recovery is a series of decisions and actions

Decide on “target recovery point”

- Restore to yesterday: Lose all today's legitimate transactions**
- Restore to just before the bug surfaced: Restore point still includes some bad data**
- Restore to now: Keeps all bad data (no help)**

Restore vs Recovery

Actual recovery is a series of decisions and actions

Reapply legitimate changes

- **Manually re-enter valid orders/transactions from today**
- **Replay selected clean transactions from logs or audits**

Handle business rule changes

- **Reapply today's schema/reference data updates after restore**
- **Or roll back the rule change temporarily**

Restore vs Recovery

Actual recovery is a series of decisions and actions

External reconciliation

- Notify customers/partners of corrections**
- Issue credit notes, reverse shipments, adjust external systems**

Communication & downtime

- How long can the system be offline?**
- Do we use read-only mode during recovery?**

Restore vs Recovery

Restore gets you a technically consistent database from the past.

Recovery answers: “Is this past state actually usable for the business right now?”

Often the answer is “not directly” — requiring additional work to bridge the gap between the restored data and current business reality.

Anyone with backup access can perform a restore.

True recovery requires coordination between DBAs, application teams, and business stakeholders to return to a state that is both technically sound and operationally correct.

RPO & RTO

RPO (Recovery Point Objective) - The maximum amount of data loss (in time) that the business can tolerate.

- How much data (measured in time) are you willing to lose?**
 - Example: RPO = 4 hours means yYou can tolerate losing up to 4 hours of transactions**
- Setting up an RPO requires frequent backups or replication to minimize data loss**

RPO & RTO

RTO (Recovery Time Objective) - The maximum acceptable downtime before the system must be back online.

- How long can the business be offline before severe impact?**
 - Example: Setting RTO as 1 hour means the entire recovery process (detect, decide, restore, verify) must complete within 1 hour**
- Drives speed of restore procedures and automation**

RPO & RTO

Backup frequency is driven by RPO

Backup type is driven by both

- **Tight RPO favors incremental.**
- **Tight RTO favors full.**

Cost is driven by both

- **Tight RPO/RTO means higher cost**

Complexity is driven by both

- **Tight RPO requires Continuous Data Capture**
- **Tight RTO requires Automated Failover**

Does GFS work with Tight RPO & RTO?

Traditional GFS is not ideal for tight (aggressive/low) RPO and RTO requirements.

- It works well for moderate tolerances, but struggles when you need minutes or seconds of data loss/downtime. GFS is acceptable for RPO of hours-to-1 day (incremental every hour/day). But it becomes hard to sustain for $RPO < 30$ min.**
- A restore requires Full backup + chain of incrementals + possibly logs. Sequential application increases time. Typically hourly chain + daily full would take up to an hour to restore.**

Synthetic Full

A synthetic full is a full backup that is constructed by combining a previous full backup with subsequent incremental backups — without reading the entire live database again.

It acts like a regular full backup (standalone, no chain dependency) but is built from existing backups.

Synthetic Full

How It Works (Conceptually)

- Start with an existing base full backup.
- Add all incremental backups taken since that base full.
- Merge them into a new, consolidated full backup that represents the database state as of the latest incremental.
- The result: A brand-new full backup that can be used independently for restores.

Synthetic Full

Benefits (Trade-offs Summary)

- **Time:** Much faster than a traditional full (no full DB scan).
- **Safety:** Produces a standalone full; so there are shorter restore chains, faster RTO, lower risk of chain breakage.
- **RPO/RTO Fit:** Excellent for tighter objectives as frequent synthetic fulls keep chains short while keeping backup windows small.
- **Storage Efficiency:** Better than repeated traditional fulls (reuses incrementals).
- **Complexity:** Higher on the backup system side (requires merge capability), but simpler for restore operations.

Synthetic Full

Resource usage of synthetic full backups

- **CPU** : Moderate-to-high use in between full/incremental backups. Merges and consolidates backup files; decompression + re-compression often needed. But this works in the background. You need to allocate just enough resources to create synthetic full before the next incremental.
- **Memory**: Moderate use. This is again because this is a background process.
- **Disk I/O**: High as there are a lot of reads and writes at the same time.
- **Impact on Live DB**: None as these operations are supposed to take place on a different physical server.

Synthetic Full

While synthetic full backups are widely supported and commonly used in many modern enterprise-grade tools (especially for VM/file system backups), not all tools support them.

Support varies by tool, workload type (e.g., VMs vs. databases), and architecture.

- Synthetic fulls are popular in virtualization and general-purpose backup tools to enable "forever incremental" strategies with periodic consolidated fulls.**
- Many database-native tools focus on full/incremental/log backups instead.**
- Some modern tools use alternative approaches (e.g., snapshot chaining) rather than traditional synthetic merges.**

Synthetic Full

Why synthetic full backups are rarely used for relational databases:

- RDBMS backups focus on transaction log backups for precise recovery to any second/minute. Synthetic fulls don't integrate well with continuous log shipping — databases prefer full + incremental + logs applied during restore.**
- Many databases have built-in mechanisms that achieve short chains and fast recovery without synthetic merges. Therefore DBAs do not need synthetic fulls.**
- Database backups are often streamed backup sets (compressed, encrypted blocks). Merging them synthetically requires complex decompression/recompression — inefficient and error-prone. VM/file tools use block-level backups easier to merge offline.**

Synthetic Full

Why synthetic full backups are rarely used for relational databases:

- Database tools verify backups during creation (e.g., block checksums). Synthetic merges on secondary storage skip live DB validation, risking undetected corruption.**
- Many enterprise backup apps explicitly do not support synthetic fulls for database agents — they convert them to traditional fulls. Database-specific logic doesn't align with offline synthesis.**
- Synthetic fulls shift load to backup storage, but databases already minimize production impact via native incrementals/logs. Adding synthetic logic adds complexity without clear gains over existing PITR flows.**

Synthetic Full

Summary:

- Relational databases prioritize granular, log-based PITR over synthetic consolidation.
- Native tools use full + incremental/differential + logs (or incrementally updated copies) because it's simpler, more reliable, and perfectly suited to transactional consistency needs.
- Synthetic fulls shine in VM/file backups but don't fit the RDBMS recovery model well.

Backup & Recovery for NoSQL

NoSQL backup strategies differ from relational DBs and VMs due to distributed architecture, eventual consistency, and built-in replication.

They have varying support on traditional backups.

- Data spreads across nodes; consistency challenges (eventual consistency) make offline merging risky without coordination.**
- There are many built-in alternatives such as native snapshots, replication as hot backup, etc.**

They often do not support synthetic fulls.

Backup & Recovery for Cloud Hosted DBs

Cloud-hosted databases (managed services like RDS, Aurora, Cloud SQL, Azure SQL, DynamoDB) shift much of the backup and recovery responsibility to the cloud provider (based on your CSP & contract).

Based on DB, CSP and contract you can get:

- Automated Backups**
- Point-in-Time Recovery (PITR)**
- Synchronous replicas in different availability zones**
- Async replicas or global tables (cross-region)**
- Snapshots & Exports**

Assisting Backup & Recovery

Combine with other safeguards

- Application-level validation**
- Change data capture / auditing trails**
- Anomaly detection monitoring**
- Immutable logs or write-once storage for critical data**
- Data Quality**
- Dual Entry Systems**
- Authoritative Sources**

Assisting Backup & Recovery

Application-level validation

- Business rules and checks implemented in the application code (or API layer) before data reaches the database.
- Catches semantically wrong data before it ever gets committed.
- Example: Verify that a new customer's email is unique across the system and matches a known domain.

Assisting Backup & Recovery

Change data capture / auditing trails

- Comprehensive logging of who changed what, when, and from/to what values.
- Built-in audit tables, database audit features (e.g., Oracle Audit, SQL Server Audit), or CDC tools (Debezium, GoldenGate).
- Enables forensic analysis when wrong data is discovered.
- Allows selective rollback of bad changes without full restore.
- Supports compliance and root-cause investigation.

Assisting Backup & Recovery

Anomaly detection monitoring

- Automated checks for unusual patterns in data or operations.
- Examples:
 - Daily job checking that total account balances match expected control totals.
 - Alerts when inventory goes negative or sales spike abnormally.
 - Statistical monitoring (e.g., sudden drop in average order value).
 - Data quality dashboards tracking null rates, duplicates, format violations.

Assisting Backup & Recovery

Immutable logs or write-once storage for critical data

- Never update or delete data — only append new events**
- Full history is preserved; you can reconstruct any past state.**
- Wrong updates can be corrected by appending compensating events.**
- Makes auditing trivial.**
- Higher complexity and storage needs. In larger applications, immutable systems are over-engineered dreams.**

Assisting Backup & Recovery

Data Quality Tools and Profiling (We'll get to that later)

- Regular scanning of data for inconsistencies using specialized tools (Talend, Informatica, Great Expectations, dbt tests).
- Proactively identifies drifting or corrupted data.
- Common checks:
 - Referential integrity beyond FKs (soft links).
 - Consistency across related tables.
 - Freshness (data not too old).
 - Cardinality and distribution checks.

Assisting Backup & Recovery

Dual Entry / Approval Workflows

- Critical data requires entry or approval by two people (common in finance).
- Reduces human error on high-impact fields (e.g., bank transfers, medical records).

Assisting Backup & Recovery

Integration with Authoritative Sources

- Regularly syncing or validating against trusted external systems.
- Keeps data aligned with ground truth.
- Examples:
 - Customer data synced with CRM.
 - Product prices pulled from master catalog.
 - Addresses validated against postal service APIs.

Thanks